Supplementary Material for "MERIT: LLM-Evolved Interpretable Bidding-and-Dispatch Programs for Virtual Power Plants"

This document collects the nomenclature, the verbatim artifacts and the full tables behind the main text. Section, figure, table and equation numbers without an S prefix refer to the main text.

NOMENCLATURE

Sets and indices:

$n \in \mathcal{N}$	Market nodes of the VPP, each mapped to a feeder bus $m(n)$
$t \in \mathcal{T}$	Hourly dispatch intervals of the operating day, $\lvert\mathcal{T}\rvert = T$
d	Operating day; $\mathcal{D}_{\mathrm{tr}}$ and $\mathcal{D}_{\mathrm{te}}$ denote the training and held-out days
k	Forecast-error scenario index; $\mathcal{S}$ is the frozen set of training pairs (d, k)

Parameters:

$p_{n,t}$	Nodal electricity price, known day-ahead (CNY/kWh)
$\bar{p}_t$	Mean nodal price at hour t
$\hat{w}^{\mathrm{pv}}_{n,t},\ \hat{w}^{\mathrm{wd}}_{n,t}$	Forecast PV and wind output (kW)
$w^{\mathrm{pv}}_{n,t},\ w^{\mathrm{wd}}_{n,t}$	Realized PV and wind output (kW)
$c^{+},\ c^{-}$	Over- and under-delivery penalty multipliers
$\kappa_{\mathrm{s}},\ \kappa_{\mathrm{v}}$	Degradation cost of storage and EV throughput (CNY/kWh)
$E^{\mathrm{s}}_n,\ P^{\mathrm{s}}_n,\ \eta_n$	Storage energy capacity, power limit and efficiency at node n
$E^{\mathrm{v}}_n,\ P^{\mathrm{v}}_{n,t}$	EV virtual-battery capacity and hourly available power
$\alpha,\ \lambda$	CVaR tail level and risk weight of the fitness
$\underline{v},\ \overline{v}$	Squared voltage limits of the feeder buses
κ	Scale factor mapping the city-scale fleet onto the feeder

Decision and state variables:

b_t	Day-ahead city-level bid (kW)
$x_{n,t}$	Net injection of node n (kW, positive into the grid); $y_t = \sum_n x_{n,t}$ is the city-level delivery
$o_t,\ u_t$	Over- and under-delivery at the city level (kW)
$a_{n,t}$	Action ratios of node n returned by the program
$\varphi_d,\ \psi_{n,t}$	Day-ahead and hourly feature vectors seen by the program
$e^{\mathrm{s}}_{n,t},\ e^{\mathrm{v}}_{n,t}$	Storage and EV energy states (kWh)
β_t	Voltage-gate retention factor applied to all actions at hour t
π	Daily net profit; $\pi_{d,k}$ its value on scenario (d, k)
F	Risk-adjusted fitness of a candidate program P
$[z]^{\pm}$	Positive and negative parts of z, $[z]^{+} = \max(z, 0)$ and $[z]^{-} = \max(-z, 0)$

S-I. PROMPT AND FEATURE KEYS

The system prompt below is the static prefix sent in every evolution call, exactly as rendered from the evaluator configuration (Section III-B), so the constants it states are the ones the evaluator applies. The two key lists inside it are the day-ahead and hourly feature keys of Section II-D, and every listed key is produced by the evaluator. The prefix states general facts of the market and the physics and contains no bidding rule, no hedging formula and no dispatch heuristic.

```
You are an expert power-market algorithm engineer. Improve
  a Python program that operates a
virtual power plant (VPP) portfolio in a two-settlement
  electricity market. You will see the
current program and its measured performance breakdown;
  rewrite it to earn more risk-adjusted
profit. Output ONE complete Python program in a single ```
  python code block.

## The three functions you must provide
1. `make_bid(day_feat) -> list[24]` -- day-ahead city-level
   bid (kW per hour), decided once
    before the operating day using forecasts only.
2. `alpha_score(feat) -> float` -- hourly per-node
  allocation score; city bid quantity is split
    across 15 nodes by softmax of these scores.
3. `device_allocation(target_kw, feat) -> dict` -- hourly
  per-node device commands:
    `pv_ratio, wind_ratio ∈ [0,1]`; `storage_ratio,
  vehicle_ratio ∈ [-1,1]` (positive =
    discharge to grid, negative = charge); `ac_ratio ∈
  [0,1]`; `wash_on_number` (int ≥ 0).

## day_feat keys (make_bid; 24-length arrays unless noted)
`pv_pred, wind_pred` (city forecast kW), `price` (CNY/kWh
  city mean), scalars `price_p30,
price_p70`, forecast-error day-quantile curves `resid_q10,
  resid_q50, resid_q90` (kW, from
historical prediction residuals), fleet: `storage_cap,
  storage_pmax, veh_cap` (scalars, kWh/kW),
`veh_pmax` (24-array), `deg_storage, deg_vehicle` (CNY/kWh
  throughput), `T` (=24).

## feat keys (alpha_score / device_allocation; per node,
  hour t)
`price, price_p30, price_p70, pv_pred, wind_pred,
  storage_soc_frac, vehicle_soc_frac,
storage_pmax, veh_pmax, total_bid, hour, bus_depth,
  beta_last`.

## Market and physics facts (settlement identical to the
  evaluator, auto-rendered)
- Profit = Σ price·delivered - Σ p̄(t)·[0.5·over(t) + 1.5
  ·under(t)] - degradation.
   Under-delivery is penalized 3x as heavily as over-
  delivery. p̄(t) = hourly mean price.
- Fitness = E[profit] - 1.0·CVaR_0.05(loss) across scenario
    worlds drawn from
   historical forecast errors: risk of bad days matters, not
    only the average.
- Storage/EV: SoC dynamics with efficiency and hourly power
    limits (`veh_pmax` varies by hour);
   degradation cost 0.05/0.08 CNY per kWh throughput (
  storage/EV). AC and wash machines only
   consume power.
```

```
- A distribution-network gate scales down all node
  injections by β ≤ 1 in hours where voltage
  limits would be violated; clipping happens more easily at
   electrically deep buses
  (feature `bus_depth`; `beta_last` is the previous hour's
  β). Clipped energy earns nothing.
- Renewable forecasts are imperfect: actual output differs
  from `pv_pred/wind_pred`; the
  residual quantile curves describe how wrong forecasts
  have historically been. Flexible
  devices can compensate errors in real time within their
  power and energy limits.

## Hard rules
- Only `numpy` (as `np`) and `math` are available; no other
   imports, no I/O, no globals.
- Functions must be deterministic and fast; never raise on
  any valid input.
- Keep the whole program under ~120 lines with minimal
  comments.
- Bids are clipped by the evaluator to [0, physical
  delivery capability]; NaN → 0.

## What the score report means (attached to the current
  program)
`revenue` (gross energy income), `dev_over_cost`/`
  dev_under_cost` (deviation penalties),
`deg` (degradation spend), `beta_binding` (share of hours
  clipped by the network gate),
`dev_mwh` (absolute deviation). Reduce the costs that
  dominate; keep revenue.
```

The two re-evolution islands of Sections V-D and V-F used this prefix, the second with the residual quantiles removed from both the prompt and the feature dictionary, and its program references none of them. Both used the same model alias, which the provider resolved to deepseek-flash rather than the snapshot of Table I. Each call appends a dynamic tail that carries the parent program in full and an itemized bill of its evaluation. The tail below is the one generated with the champion program as parent, on the training scenarios of Section III-C, where the gate binds in 20.5% of the hours against 4.2% on the test days (Section V-E), with the program elided.

```
Current program:
```python
...  # the parent program in full
```

Measured performance (mean per episode, CNY):
fitness=9,654,660 E[profit]=6,348,520 worst=3,250,732
  beta_binding=20.5% dev=3,154 MWh
- revenue: 6,923,743
- dev_over_cost: 369,906
- dev_under_cost: 169,211
- deg: 36,106

Rewrite the FULL program to increase fitness. One ```python
   block only.
```

S-II. Champion Program

The champion program is reproduced line by line. Fig. 5 excerpts the bid and allocation-score functions with the four mechanisms marked. The device-allocation function, omitted there, appears here in full. The comments are the model's own and are not always accurate. The final rule of the bid function lowers the bid, which reduces under-delivery rather than over-delivery.

```python
import numpy as np

def make_bid(day_feat):
    pv, wind = day_feat["pv_pred"],
    ↪ day_feat["wind_pred"]
    price = day_feat["price"]
    base = pv + wind
    storage_cap = day_feat["storage_cap"]
    storage_pmax = day_feat["storage_pmax"]
    veh_cap = day_feat["veh_cap"]
    veh_pmax = day_feat["veh_pmax"]

    # Reduce under-delivery by using robust reserve:
    ↪ start with q50 and trim downward
    bid = base + 0.5 * day_feat["resid_q50"] + 0.3 *
    ↪ day_feat["resid_q10"]

    # Time-of-use price shaping: only lift high-price
    ↪ hours, avoid over-committing during low
    p_med = np.median(price)
    p_high = p_med * 1.15
    p_low = p_med * 0.85
    bid = np.where(price > p_high, bid * 1.12, bid)
    bid = np.where(price < p_low, bid * 0.92, bid)

    # Storage arbitrage: charge 3 cheapest, discharge
    ↪ 3 most expensive
    price_rank = np.argsort(price)
    charge_h = price_rank[:3]
    discharge_h = price_rank[-3:]
    energy = min(storage_cap * 0.5 * 0.9,
    ↪ storage_pmax * 3)
    charge_per = energy / 3
    for h in charge_h:
        bid[h] = max(0, bid[h] - charge_per * 0.9)
    for h in discharge_h:
        bid[h] += charge_per * 0.85

    # Vehicle arbitrage: only 3 hours each way,
    ↪ smaller fraction
    veh_energy = min(veh_cap * 0.3, np.mean(veh_pmax)
    ↪ * 3)
    veh_per = veh_energy / 3
    for h in discharge_h[:3]:
        bid[h] += veh_per * 0.8
    for h in charge_h:
        bid[h] = max(0, bid[h] - veh_per * 0.9)

    # Slightly reduce extreme bids to avoid
    ↪ over-delivery penalty
    bid = np.where(bid > 1500, bid * 0.95, bid)
    return list(np.clip(bid, 0, None))

def alpha_score(feat):
    price = feat["price"]
    p70 = feat["price_p70"]
    depth = feat["bus_depth"]
    beta = feat["beta_last"]

    # Strong price gradient, but also penalize deep
    ↪ buses heavily
    price_factor = 1.0 + 30.0 * max(0, price - p70) /
    ↪ max(1e-6, p70)
    depth_factor = 1.0 / (1.0 + 2.5 * depth)
    beta_factor = 0.3 + 4.0 * beta
    soc_factor = 0.2 + 0.8 * feat["storage_soc_frac"]
    ↪ + 0.1 * feat["vehicle_soc_frac"]
    return max(1e-6, price_factor * depth_factor *
    ↪ beta_factor * soc_factor)

def device_allocation(target_kw, feat):
    price = feat["price"]
    p30, p70 = feat["price_p30"], feat["price_p70"]
    pv_pred, wind_pred = feat["pv_pred"],
    ↪ feat["wind_pred"]
    storage_pmax = feat["storage_pmax"]
    veh_pmax = feat["veh_pmax"]
    soc_s = feat["storage_soc_frac"]
    soc_v = feat["vehicle_soc_frac"]

    pv_ratio = 1.0
    wind_ratio = 1.0

    # Storage: more careful thresholds, avoid deep
    ↪ cycling
    s = 0.0
    if price >= p70 * 0.9 and soc_s > 0.05:
        s = min(1.0, soc_s * 3.0)
    elif price <= p30 * 1.1 and soc_s < 0.95:
        s = -min(0.8, (1-soc_s) * 3.0)

```

```
    # Vehicle: only when price is very high, smaller magnitude
    v = 0.0
    if price >= p70 * 1.1 and soc_v > 0.1 and veh_pmax > 0:
        v = min(0.9, soc_v * 2.5)
    elif price <= p30 * 0.8 and soc_v < 0.9 and veh_pmax > 0:
        v = -min(0.7, (1-soc_v) * 2.5)

    # Error compensation: be less aggressive to prevent over-committing
    diff = target_kw - (pv_pred + wind_pred)
    if diff > 0:
        if s > 0 and soc_s > 0.05:
            s = min(1.0, s + diff / max(1e-6, storage_pmax) * 0.6)
        if v > 0 and soc_v > 0.1 and veh_pmax > 0:
            v = min(0.9, v + diff / max(1e-6, veh_pmax) * 0.3)
    elif diff < 0:
        if s < 0 and soc_s < 0.95:
            s = max(-1.0, s + diff / max(1e-6, storage_pmax) * 0.5)
        if v < 0 and soc_v < 0.9 and veh_pmax > 0:
            v = max(-0.7, v + diff / max(1e-6, veh_pmax) * 0.3)

    # AC and wash only at extreme low prices, small effect
    ac_ratio = 0.0
    wash_on_number = 0
    if price <= p30 * 0.2:
        ac_ratio = 1.0
        wash_on_number = 5
    elif price <= p30 * 0.35:
        ac_ratio = 0.6
        wash_on_number = 3

    return {"pv_ratio": pv_ratio, "wind_ratio": wind_ratio,
            "storage_ratio": s, "vehicle_ratio": v,
            "ac_ratio": ac_ratio, "wash_on_number": wash_on_number}
```

S-III. Forecast-Error Scenario Generator

Every scenario is a whole-day residual applied to the forecast of the target day. For each training day j the residual $r^{\mathrm{src}}_{n,j,t} = w^{\mathrm{src}}_{n,j,t} - \hat{w}^{\mathrm{src}}_{n,j,t}$ is stored per node, hour and source. Scenario k of day d draws one training day j_k uniformly and applies its residual to all nodes and both sources at once,

$$\tilde{w}^{\mathrm{src}}_{n,t}(d,k) = \max\left(\hat{w}^{\mathrm{src}}_{n,d,t} + s\, r^{\mathrm{src}}_{n,j_k,t},\, 0\right),$$

with s the amplitude scale of Table IV and $s = 1$ elsewhere. The residual atoms are the 5 training days, so no scenario contains a test-day residual. The frozen set of Section III-C draws 5 scenarios per training day with replacement, 20 distinct day-residual combinations among 25 episodes, and the recheck set of Section III-D is a second such draw. The test set of Section IV-C and the scale study of Table IV enumerate all 5 atoms for each held-out day, 10 scenarios per scale. The stochastic bid of Section IV-B draws 25 per day from its own seed. Every method is run on the same scenarios. In the scale study the residual quantile features of the programs and the shading quantiles of the references stay at their empirical values, so scales other than one test errors that every method misjudges.

S-IV. Evolution Under Four LLM Providers

Table S-I repeats the evolution under three further providers with the identical protocol, that is, the same prompt verbatim, the same program contract, the same frozen scenario set and the same adjudication rule. The champion of the main text was frozen before these runs and was not reselected. The reasoning-mode row is listed because that variant returned no complete program at the budget given in the note. The base seed bids the forecast sum and delivers half of the renewable output with storage and EVs idle. The arbitrage seed keeps that bid and adds a price-quantile rule, discharging storage and EVs when the price is above its daily seventieth percentile and charging below the thirtieth.

S-V. Information Available to Each Method

Table S-II lists what each method sees day-ahead and in real time, what it is tuned on and where its bid comes from. Two asymmetries favor the baselines, since the optimization planners see whole-day forecasts and MPC-nc also sees the actual renewable output of the current hour. One favors the programs, which see the electrical depth of their bus and the retention factor of the previous hour. Table S-IV measures what that is worth.

S-VI. Paired Tests

Table S-III lists all eight comparisons fixed before the tests on the 10 scenarios, with exact tests, including the two expert-shaded references, whose profit differences from MERIT are not significant. The main text quotes the comparisons with MPC-nc, the transplant and the two expert-shaded references. The hand-written program was added after the comparisons were fixed and is outside this list.

S-VII. Attribution of the Gate Features

Table S-IV re-evaluates the champion with the two gate features replaced by constants in every hourly feature vector, a mid-depth bus and an unclipped previous hour, with the bid function unchanged. Test profit, fitness and deviation move by under a tenth of a percent, so the gate features shape the allocation rule shown in Fig. 5 without carrying measurable profit on these days.

S-VIII. Full Parameter Table

Table S-V lists every parameter of the market, the network, the fleet, the data and the evolution; Table I of the main text is its subset.

S-IX. Second Held-Out Day

Fig. S1 repeats the panels of Fig. 4 for the second held-out day. The day differs in profile, with a midday dip in renewables, a bimodal voltage envelope, a champion that turns from over- to under-delivery at noon, and a jagged storage state of charge before dawn. The three patterns of Fig. 4 nevertheless hold. The bid tracks the renewable profile, storage and EVs cycle between cheap and expensive hours, and both envelopes press against the upper limit without exceeding it.

TABLE S-I
EVOLUTION UNDER FOUR LLM PROVIDERS

Provider and model	Seed	Best training fitness (M CNY)	Test profit (M CNY)	Test |dev| (MWh)	API cost (CNY)	Valid-mutation rate
DeepSeek deepseek-v4-flash (main campaign)	base, arbitrage	9.655 (islands 9.410–9.655)	8.471	2823	10.06	90%
Alibaba qwen3.8-flash	base	9.405	8.376	3134	2.17	89%
Alibaba qwen3.8-flash	arbitrage	9.375	8.432	3011	2.21	95%
Zhipu glm-4.5-air	base	7.320	6.904	6719	2.38	80%
Zhipu glm-4.5-air	arbitrage	9.450	8.541	2996	2.17	77%
OpenAI gpt-4o-mini	base	5.874	5.628	12005	2.18	92%
OpenAI gpt-4o-mini	arbitrage	8.928	8.053	5371	1.87	72%
DeepSeek deepseek-reasoner (reasoning mode)	arbitrage	— (no valid program)	—	—	2.2 (aborted)	0%

Note: the selection metric is the training fitness of Section III under the frozen protocol; the expert-shaded reference evaluated under the same protocol scores 9.468 M CNY, and the main-campaign champion was frozen before these runs and not reselected. 4 of the 6 islands of the three further providers, including every arbitrage-seed run, beat every classical planner on the test days (MPC-nc 7.467 M CNY) by 7.8 to 14.4%, and Zhipu glm-4.5-air (arbitrage) lands inside the main-campaign island range on training fitness. GLM tokens were covered by a free grant (nominal cost shown); the reasoning-mode row exhausted 8k-token calls over 14 generations and a single 32k-token call without a complete program.

TABLE S-II
INFORMATION AVAILABLE TO EACH METHOD

Method	Day-ahead information	Real-time information at hour t	Training or tuning	Bid
Naive	none	node features (local prices with daily quantiles, forecasts, own states of charge, fleet limits)	none	exogenous bid
SAC-RL	none	city-level observation (hour, mean price with daily quantiles, city forecasts, bid, aggregate states of charge)	training days with residual-bootstrap augmentation; 8 configurations $\times$ 3 seeds, selected on training days	exogenous bid
DET	whole-day forecasts, prices, fleet parameters	none (open loop)	none	exogenous bid
MPC-nc	as DET	actual states of charge; actual renewable output of the current hour	none	exogenous bid
DET-bid, MPC-bid	as DET, MPC-nc	as DET, MPC-nc	none	day-ahead LP bid, frozen
DET-shaded, MPC-shaded	as DET-bid, MPC-bid, plus training-day residual quantiles	as DET, MPC-nc	shading quantile τ tuned on training days over a grid fixed before tuning	LP bid shaded by the residual quantile, frozen
SP-bid	forecasts plus 25 residual-bootstrap scenarios of the training days (seeds distinct from the evaluation scenarios)	as MPC-nc	none (λ, α from the shared configuration)	two-stage stochastic bid, frozen
Hand-written program	forecasts, price statistics, training-day residual quantiles, fleet parameters	node features (local prices with daily quantiles, forecasts, own states of charge, fleet limits)	none	newsvendor hedge at the critical fractile, frozen
MERIT	forecasts, price statistics, training-day residual quantiles, fleet parameters	node features (bus depth, previous-hour β, local prices with daily quantiles, forecasts, own states of charge, fleet limits)	training days $\times$ 5 frozen scenarios; champion frozen before any test evaluation	evolved bid function, frozen day-ahead
Oracles	realized whole-day output (perfect foresight; bounds, not methods)	—	none	exogenous / free

Note: every executed method passes through the same voltage gate, dispatch chain and settlement, starts from identical states of charge, knows the day-ahead prices, and never observes future realizations; all tuning and selection use the training days only. Known asymmetries: the optimization planners see whole-day forecasts while programs see hourly features; MPC-nc additionally sees the actual renewable output of the current hour; programs additionally see the gate features.

TABLE S-III
PAIRED WILCOXON TESTS ON THE TEST SCENARIOS

Comparison (paired over the same scenarios)	Metric	Wins	Median paired difference	p (raw)	p (Holm)
MERIT vs MPC-nc	profit (MERIT higher)	10/10	+1.076 M CNY	9.8e-04	0.003
MERIT vs DET-shaded	profit (MERIT higher)	6/10	+0.047 M CNY	0.500	1.000
MERIT vs MPC-shaded	profit (MERIT higher)	5/10	-0.015 M CNY	0.839	1.000
MERIT vs MPC-nc	|deviation| (MERIT lower)	10/10	+3457 MWh	9.8e-04	0.003
MERIT vs DET-shaded	|deviation| (MERIT lower)	4/10	-53 MWh	0.385	0.770
MERIT vs MPC-shaded	|deviation| (MERIT lower)	3/10	-174 MWh	0.839	0.839
Transplant (MERIT bid + MPC-nc dispatch) vs MERIT	profit (two-sided)	10/10 transplant higher	+0.167 M CNY	0.002	0.004
Transplant vs MERIT	|deviation| (two-sided)	6/10 transplant higher	+1022 MWh	0.193	0.193

Note: N=10 scenarios, each held-out day combined with each training-day residual, identical for every method; exact Wilcoxon signed-rank tests, one-sided for the baseline comparisons (three comparisons per metric family, Holm-corrected within the family) and two-sided for the transplant comparison; expert-shaded references at their final train-tuned quantile (DET-shaded 0.02, MPC-shaded 0.05). The main text quotes the MPC-nc, transplant and expert-shaded profit comparisons, and the remaining rows appear only here.

S-X. Leave-One-Day-Out Cross-Validation

Each held-out day is evaluated once by a champion evolved on the other six days. The residual bank, the residual-quantile features, the shading quantiles, the stochastic-bid scenarios and the fitness and recheck sets are all drawn from those six days. Each fold runs two islands, one per seed, for 400 generations, and its fitness set holds 30 episodes. The folds are not chronological, and only the fold that holds out day 7 trains on days that precede its test day. The environment constants are those of the main text, and the model snapshot returned by

TABLE S-IV
ATTRIBUTION OF THE GATE FEATURES

Program variant	Test profit (M CNY)	Fitness (M CNY)	Deviation (MWh)	Gate-binding hours (%)
Champion (gate features as evolved)	8.471	16.086	2823	4.2
Champion with gate features fixed (mid-depth bus, unclipped previous hour)	8.467	16.080	2824	4.2

Note: the two gate features (bus depth and previous-hour retention factor) are replaced by constants in every hourly feature vector and the champion is re-evaluated on the held-out days with its bid function unchanged; test profit changes by -0.06%.

TABLE S-V
FULL EXPERIMENTAL SETUP PARAMETERS

Block	Setting	Value
Market	Deviation penalty multipliers c^+ / c^- ($\times$ hourly mean price)	0.5 / 1.5
Market	Degradation cost storage / EV (CNY per kWh throughput)	0.05 / 0.08
Market	Risk parameters α / λ	0.05 / 1.0
Network	Feeder / voltage limits	IEEE-33 (LinDistFlow) / [0.95, 1.05] p.u.
Network	Background-load scale / κ_{mult} / resulting κ	0.55 / 1.75 / 0.000946
Interface	Softmax temperature θ / bid-cap coefficient	1.0 / 1.2
Fleet	Market nodes (mapped to the 15 electrically deepest feeder buses)	15
Fleet	Devices: storage / EV / AC / washer (total)	901 / 2015 / 1446 / 957 (5319)
Fleet	Storage energy / power (city total)	89.0 MWh / 26.0 MW
Fleet	EV virtual battery energy / peak available power	241.9 MWh / 62.5 MW
Fleet	AC power / washer rated power (city total)	38.5 MW / 32.63 MW
Data	Days (train / test; tuning and selection on train only)	7 (5 / 2)
Data	Native resolution $\rightarrow$ decision step	15 min $\rightarrow$ 1 h (hourly means)
Data	Nodal price range / mean (CNY/kWh)	0.000–1.500 / 0.375
Data	City renewable peak (actual, 7 days)	4440.9 MW
Data	Forecast-error residual atoms (whole-day, train only)	5
Baselines	Shading quantile τ, DET-shaded / MPC-shaded (tuned on training fitness)	0.02 / 0.05
Evolution	Islands $\times$ generations (independent, no migration)	4 $\times$ 400
Evolution	Fitness scenario set: train days $\times$ draws per day	5 $\times$ 5 = 25 episodes (seed 20260827)
Evolution	Parent selection	p=0.7 uniform over top-5, else uniform
Evolution	Recheck scenario set: seed / rejection threshold; oracle tripwire tolerance	20270901 / drop > 10%; 1 CNY
Evolution	Sandbox: static screen, restricted namespace, subprocess timeout	numpy/math only; 90 s
LLM	Model (API alias $\rightarrow$ snapshot at campaign time)	deepseek-chat $\rightarrow$ deepseek-v4-flash
LLM	Sampling temperature / max tokens	0.9 / 4096
LLM	Prompt version (verbatim in the supplementary material)	v1
LLM	Campaign: calls / tokens / cost / prefix-cache hit	1600 / 7.4 M / CNY 10.06 / 81.7%

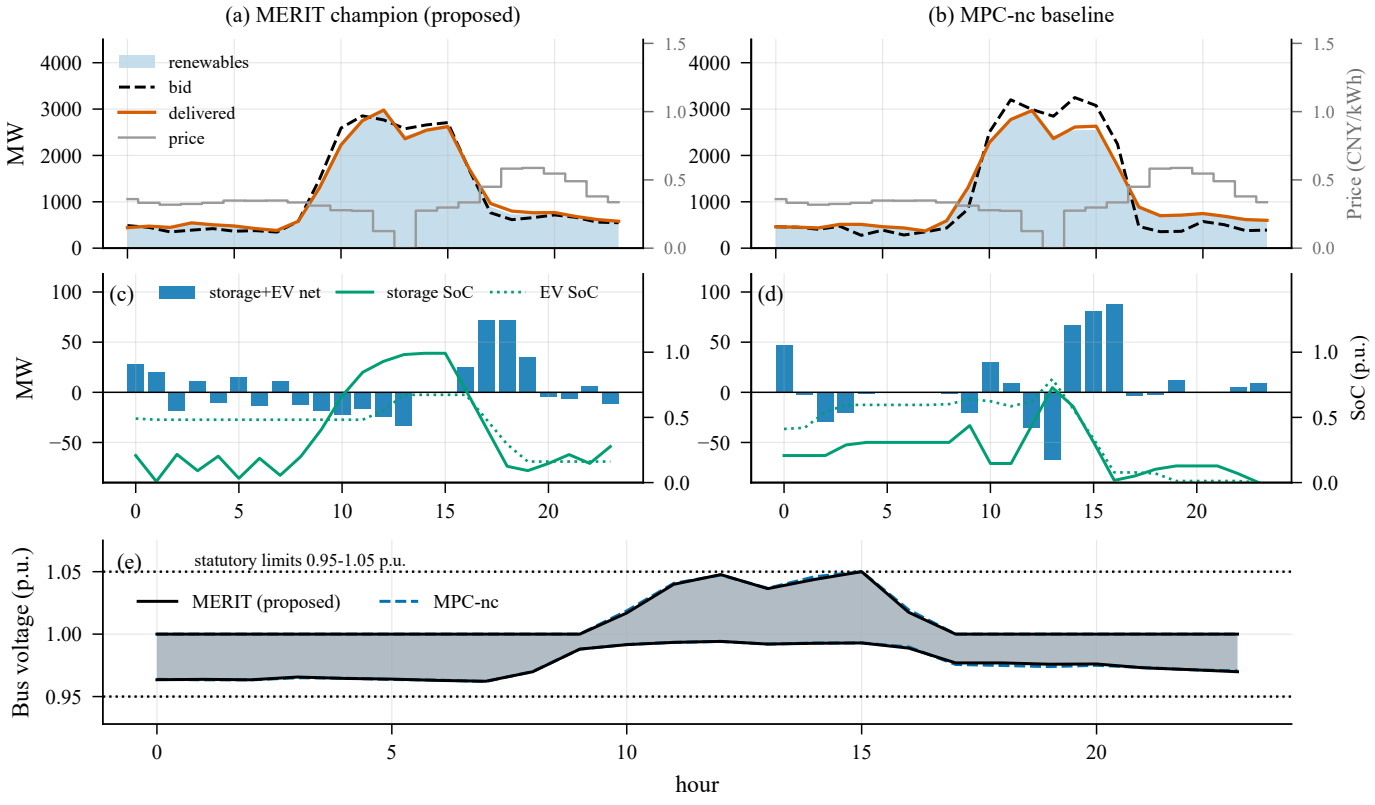


Fig. S1. Operation on the second test day; panels as in Fig. 4.

TABLE S-VI
LEAVE-ONE-DAY-OUT CROSS-VALIDATION: ALL METHODS

Held-out day	Naive	DET	MPC-nc	DET-shaded	MPC-shaded	SP-bid	Hand-written	MERIT
day 1	6.571 / 8783	6.728 / 8570	6.801 / 8384	8.699 / 3330	8.706 / 3068	7.837 / 3761	8.748 / 2346	8.225 / 4304
day 2	2.447 / 9727	2.501 / 9402	2.644 / 9049	3.126 / 2816	3.316 / 2101	3.140 / 1887	3.213 / 2977	2.899 / 4536
day 3	2.973 / 8573	3.061 / 12251	3.170 / 11999	4.334 / 3012	4.544 / 2255	4.334 / 2691	4.220 / 2927	4.315 / 3125
day 4	2.660 / 9951	2.747 / 10296	2.748 / 10283	3.410 / 2363	3.440 / 2025	3.235 / 2062	3.348 / 2878	3.378 / 2620
day 5	5.546 / 16340	5.764 / 15787	5.880 / 15484	10.184 / 3034	10.110 / 2948	10.079 / 2995	9.668 / 4248	9.853 / 3526
day 6	8.106 / 5951	8.268 / 5994	8.340 / 5488	9.682 / 3535	9.553 / 3109	8.834 / 3061	9.123 / 3896	9.562 / 2935
day 7	6.445 / 6101	6.557 / 8359	6.595 / 5794	7.294 / 4170	7.366 / 3956	7.359 / 2716	7.096 / 4281	7.583 / 2785

Cells: test-day profit (M CNY) / |deviation| (MWh); every baseline re-tuned on the fold's six training days.

TABLE S-VII
LEAVE-ONE-DAY-OUT CROSS-VALIDATION: FOLD CHAMPIONS AND TUNED QUANTILES

Held-out day	Champion island	Generation	Training fitness (M CNY)	Recheck drop (%)	τ DET-shaded / MPC-shaded
day 1	base	326	10.208	2.5	0.02 / 0.05
day 2	base	399	11.607	1.2	0.1 / 0.15
day 3	arbitrage	352	10.810	0.3	0.02 / 0.05
day 4	base	371	11.200	1.7	0.05 / 0.15
day 5	arbitrage	317	9.933	0.6	0.02 / 0.15
day 6	arbitrage	188	10.089	3.7	0.02 / 0.05
day 7	base	375	10.402	2.6	0.02 / 0.02

Generation: the generation at which the champion appeared; every island ran all 400 generations, and every API call resolved to the deepseek-flash snapshot.

TABLE S-VIII
LEAVE-ONE-DAY-OUT CROSS-VALIDATION: SIX-SCENARIO CVAR OF ALL METHODS (M CNY)

Held-out day	Naive	DET	MPC-nc	DET-shaded	MPC-shaded	SP-bid	Hand-written	MERIT
day 1	5.718	5.905	5.904	8.779	8.899	9.100	8.559	8.905
day 2	2.071	2.203	2.265	3.434	3.482	3.534	3.343	3.272
day 3	2.312	2.472	2.503	4.333	4.379	4.078	4.079	4.124
day 4	2.904	2.939	2.969	3.648	3.668	3.857	3.553	3.679
day 5	4.537	4.856	4.878	9.619	9.487	9.457	8.886	9.524
day 6	8.270	8.453	8.540	10.208	10.292	10.167	9.803	10.206
day 7	6.842	6.997	7.004	7.998	7.999	8.277	7.866	8.064

Cells: CVaR_5 of profit over the six scenarios of the held-out day, which is its worst scenario (M CNY).

the API is recorded per island. Table S-VI lists every method per fold, Table S-VII the fold champions with the tuned shading quantiles and Table S-VIII the six-scenario CVaR of every method. The exact one-sided tests give $p = 0.023$ against MPC-nc, 0.96 against MPC-shaded and 0.58 against SP-bid after Holm correction.